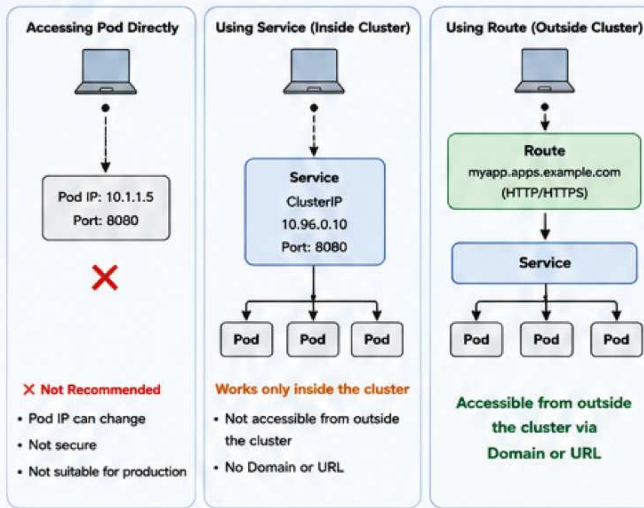


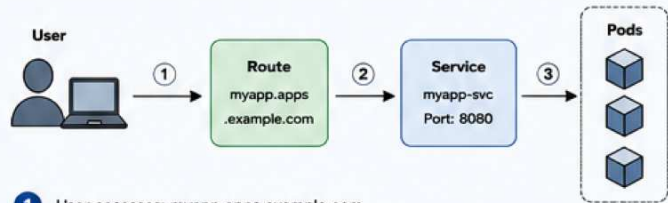
1 Introduction

- **Problem:** Accessing a Pod directly using IP and Port is difficult and not practical.
- **Role of Service:** Service provides a stable endpoint and load balances traffic to Pods.
- **Why Service alone is not enough:** Services are only reachable inside the cluster. They are not exposed to external users.
- **Need for URL or Domain:** Users expect to access applications using friendly URLs (e.g. myapp.apps.example.com) instead of IP:Port.



2 What is a Route?

- A Route is an OpenShift object.
- It maps a Domain or URL to a Service.
- It allows external users to access applications running inside the cluster.
- Difference between inside and outside access:



- 1 User accesses: myapp.apps.example.com
- 2 Route receives the request
- 3 Route forwards the request to the Service
- 4 Service load balances the request to Pods

Access Inside the Cluster

Use Service Name

Example:

http://myapp-svc:8080



Access Outside the Cluster

Use Route (Domain)

Example:

https://myapp.apps.example.com



3 Route Components (YAML)

```
apiVersion: route.openshift.io/v1
kind: Route
metadata:
  name: myapp-route
  namespace: dev-project
spec:
  host: myapp.apps.example.com
  to:
    kind: Service
    name: myapp-svc
    weight: 100
  port:
    targetPort: 8080
  tls:
    terminate: edge
    certificate: |
      -----BEGIN CERTIFICATE-----
      ...
      key: |
      -----BEGIN PRIVATE KEY-----
      ...
  path: /
  wildcardPolicy: None
status:
  ingress:
    - host: myapp.apps.example.com
      routerName: default
      conditions:
        - type: Admitted
          lastTransitionTime: "2024-01-01T12:00:00Z"
```

Field	Description
metadata	Information about the Route (name, namespace).
spec	Desired state of the Route.
host	The URL or Domain used to access the Route.
to	References the Service that will handle the request.
port	The port on the Service.
tls	TLS settings for the Route.
path	Path (optional) to match requests.
wildcardPolicy	How wildcards are handled with subdomains.
status	Current status and ingress information.

4 Types of Route

Plain Text (HTTP)

- Uses HTTP protocol.
- No encryption.



Secure (HTTPS)

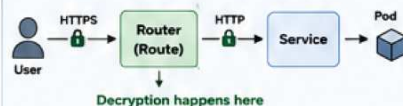
- Uses HTTPS protocol.
- Encrypted using TLS.



5 TLS Termination Types

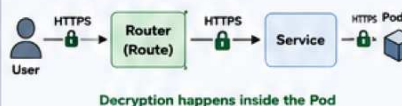
Edge (Termination at Route)

- TLS is terminated at the Route (router).
- Route decrypts HTTPS and forwards HTTP to the Service.
- Easy to use, suitable for most cases.



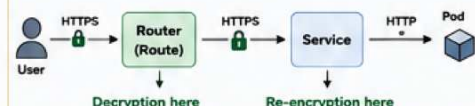
(Passthrough (Termination at Pod)

- Route passes encrypted traffic as is.
- Route does not decrypt.
- Service/Pod handles HTTPS.
- End-to-end encryption.



Re-encrypt (Termination at Both)

- Route decrypts HTTPS from user.
- Route encrypts again to the Service.
- Uses a different certificate for internal connection.



6 Practical Lab (Hands-On)

1 Create Self-Signed Certificate (Using OpenSSL)

```
openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 \
  -keyout tls.key \
  -out tls.crt \
  -subj "/CN=myapp.apps.example.com/O=DevOps"
```

Verify:
ls -ltr tls.crt tls.key

2 Create Deployment

```
oc create deployment myapp \
  --image=nginx:latest \
  --port=8080
```

Verify:
oc get pods

3 Create Service

```
oc create service clusterip \
  myapp-svc \
  --tcp=8080:8080 \
  --selector=app=myapp
```

Verify:
oc get svc

4 Create Route

```
oc create route edge myapp-route \
  --service=myapp-svc \
  --hostname=myapp.apps.example.com \
  --port=8080
```

Verify:
oc get route

5 Create Edge Route with Certificate & Private Key

```
oc create route edge myapp-secure \
  --service=myapp-svc \
  --hostname=secure.apps.example.com \
  --port=8080 \
  --cert=tls.crt \
  --key=tls.key
```

Verify:
oc get route

6 Test Access

Open in browser or use curl to test access.

```
# HTTP
curl http://myapp.apps.example.com

# HTTPS
curl -k https://secure.apps.example.com
```

Tips:

- You can update /etc/hosts to map Domain to Router IP for testing.
- Ensure the Router is reachable from outside the cluster.

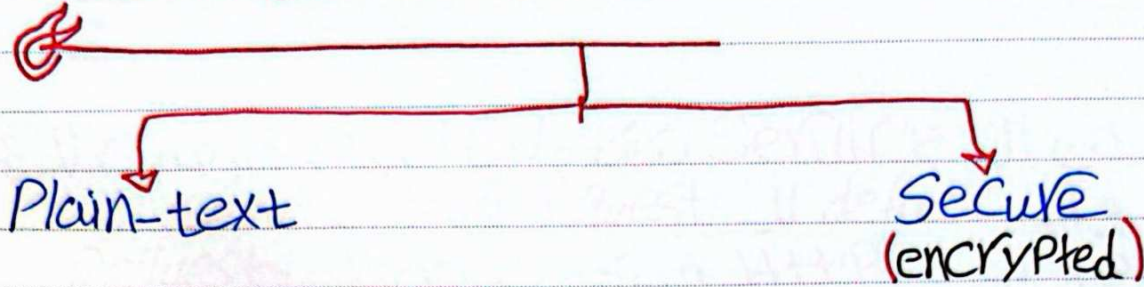
Example (/etc/hosts):

```
192.168.1.100 myapp.apps.example.com secure.apps.example.com
```

Note:

Your cluster must have an Ingress Controller (Router) exposed.

OpenShift Route



عشان أقدر أوصول لـ Pod من خلال الـ Service محتاجين $Port + IP$

- هل الـ IP بتاع الـ Server + الـ Port للـ Service دي حاجة *Readable* بالنسبة لـ User أكيد لا

- إيف قدرت اخلّي الـ Pod الناس اللي برة يفدروا يدخلوا عليها لكن لازم $Port + IP$

- أنا محتاج بغير أستبدل الـ $Port + IP$ باسم يبقى محتاج *DNS* من نوع خاص

- الـ *DNS* انما الـ *map* الـ *DNS* على الـ *IP*

① *name* بربطة بـ

② *name* بربطة بـ *IP*

③ *name* بربطة بتكثرت *IP*

- مفيش عندي امكانية تحديد الـ *Ports* في الـ *DNS*

بالتالي عشان أوصول الـ $Port + IP$ طالي *URL* عملية مرفقة جدا حاجة

- أنا أقدر أوصول الـ *IP* عادي لكن الـ *Port* لا

بالتالي الـ *User* لما رجلي يدخل يكتب *URL/Port*

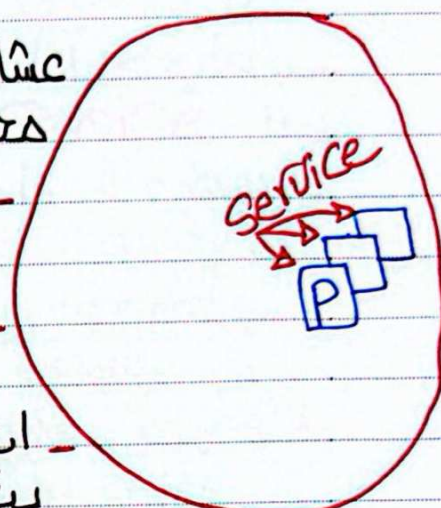
- كذا أنا لو عندي كمان أقدر من *Service* لازم ألاقي حاجة غير الـ $Port + IP$

- يبقى كذا محتاج *object* متفصّل

لأن الـ *Service* بتقدر فقط تطلع الـ *Port* لـ *User*

- كمان الـ *default* بتاع الـ *Service* شتالة بالـ *HTTP* فقط

ودي كارثة لأن كذا كل الـ *Requests* اللي طالعة *Plain text*



AP

OpenShift Rout

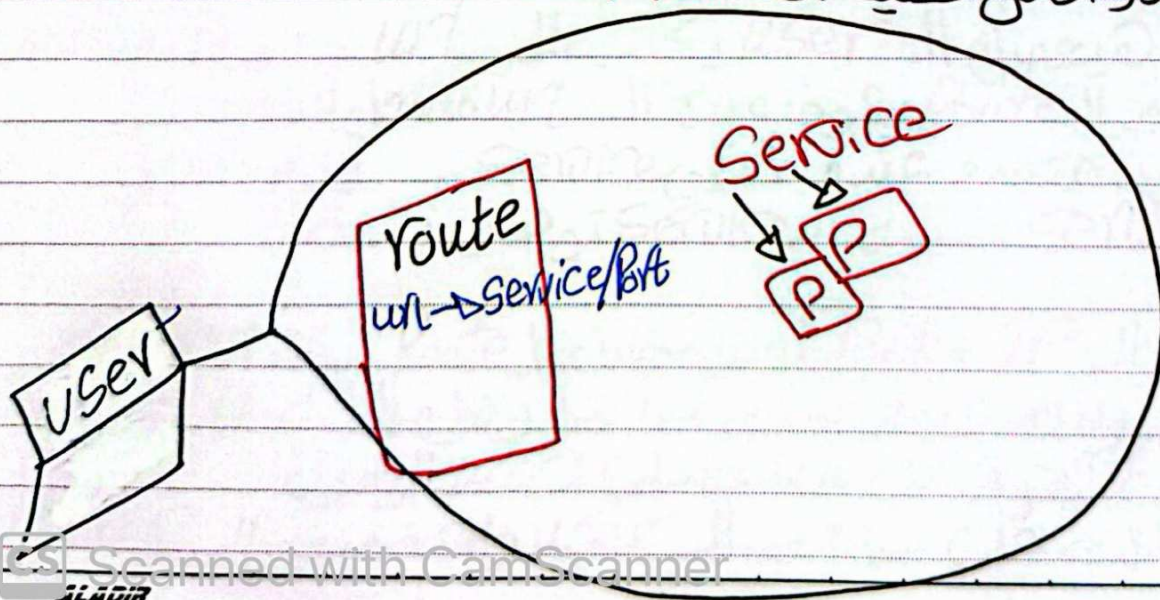
بما ان ال **Service** بتقديم اتصال خارجي لا **Port** وال **default** لا **Request** في ال **Service** هو ال **HTTP** ويكون كل ال **Requests** عبارة عن **Plain-text**

- بالتالي كذا اننا بقت محتاج تشفير
- ال **Service** فبغدها ان القدرة للتشفير ومش مهمة لكنا
- بالتالي عندها مشكلتين

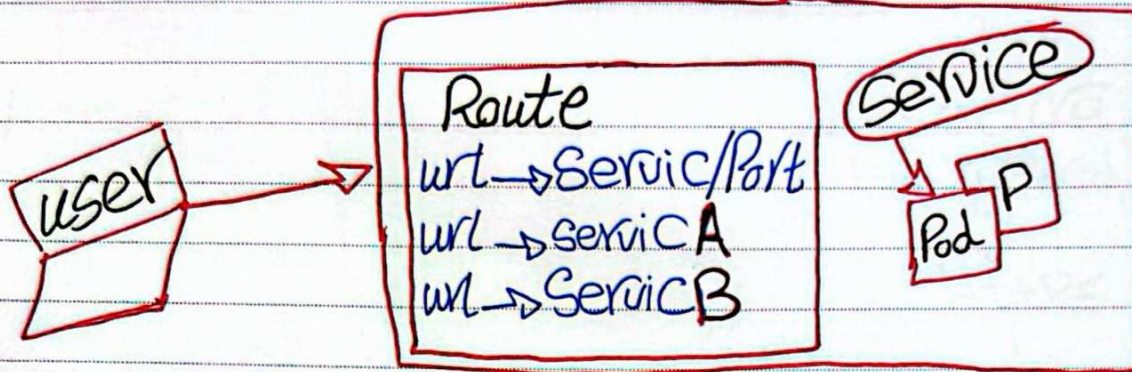
1- دايمة ال **Access** بيتر من خلال ال **IP + Port** بالتالي عايزين نستبدل داب **name Mapping** واديها ال **IP + Port** وهي تترجم

2- كمان عايزها ت **Support** ال **incryption** وتحمي كل الترافيق على مستوى ال **APP** ساعتها علولنا ال **Route**

ال **Route** عبارة عن **Object** بستخدمه عشان اخلي ال **APP** ال **Access** من برة ال **Cluster** بأستخدام ال **url** او **Name** - اقدر افعل عليه ال **TLS**



OpenShift Route



Route

meta data
spec
Host
to
Port
Tls
Path
WildcardPolicy
Status

ال Route يحتوي على

- البيانات الأساسية في اسم Route
- الجزء الذي يحدد ال Route المستغل ازاى
- ال Domain الذي يهدف الى ال user
- تحديد ال Service الذي يهدف اليها الترافيك
- يحدد ال Port الذي يستخدمه ال Route
- اعدادات ال HTTPS
- نوعين احدهم ال Route يستقبل من انهي service
- يتحدد بسماع اسمها اولاً
- ال OpenShift يحدد حالة ال Route

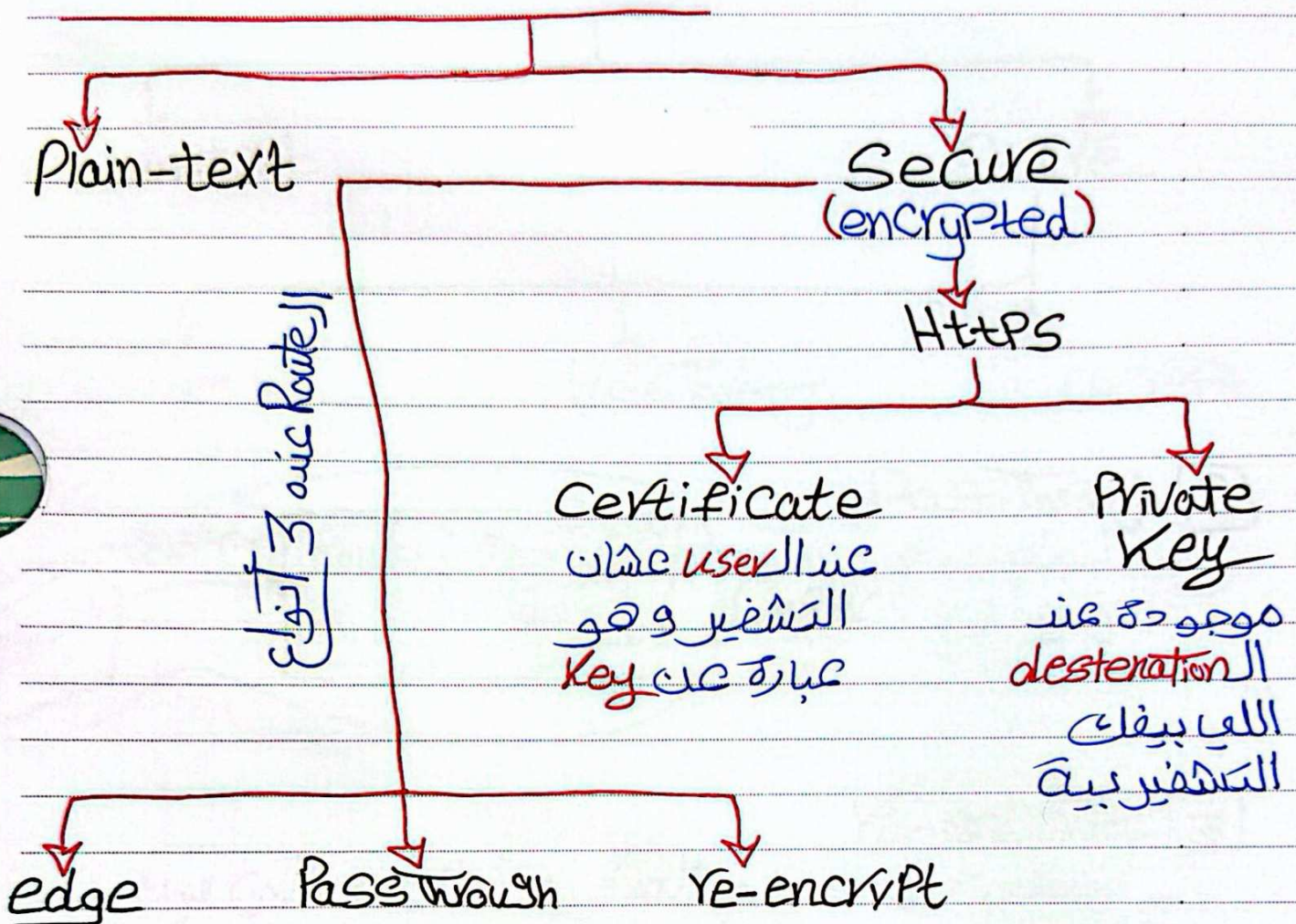
الذي يهدف الى ال User يكتب ال url

ال Route يشوف عنده ال url مسجل عنده ولا لا
محدد عنده لانهي service
ال service توصلني بال Pod

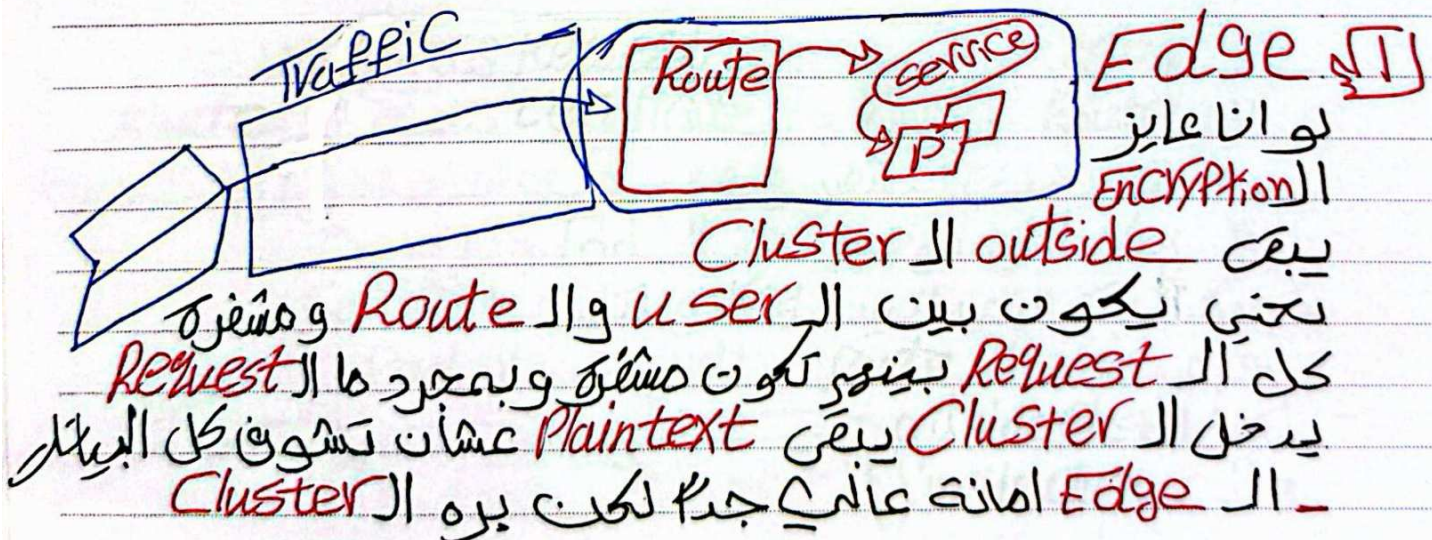
طريقة ال IP + Port أو من خلال ال Route

نتجته واحدة لأن ال User كما كذا وصل ال Pod
لكن الفرق اني سميت على ال user الوصول
حافظت على بيانات ال user واستخدمت ال TLS

OPinShift Route



كذا كل الأنواع بتستخدم HTTPS دأشوط أساسية



OpenShift Route

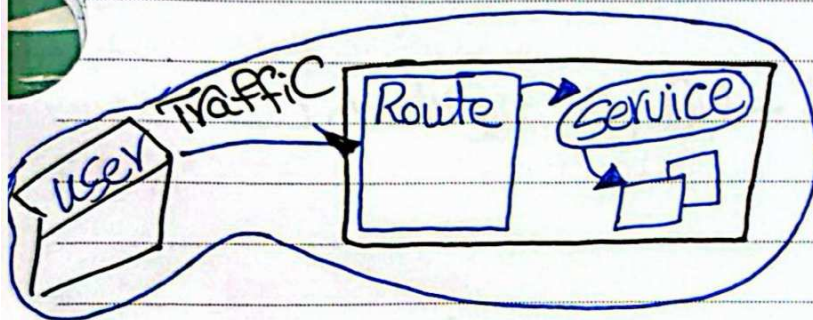
Plain-text

Secure

Edge

PassThrough

re-encrypt



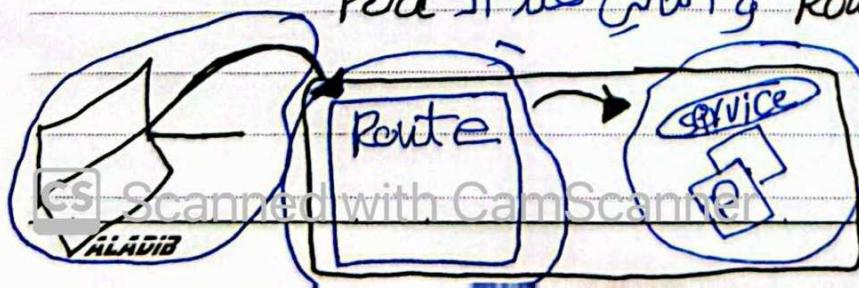
PassThrough [2]
التشفير يكون داخل وقارج الCluster

re-encrypt [3]

بمستفاد في الجوانب العسكرية أو الجوانب التي سرية
بياناتهم عالية جداً

لأنه بمستفاد Two Private Key + Two Certificate

ال user لما يبعث ال Request فتروح مشفرة
لا Route بال Key وال Certificate بتاعة ال Route
بعد ما يفلت التشفير ويفهم وبعد ما يشفر مرة ثانية
بال Key اللي موجود عند ال Pod ويبعث
ويكبر اربع عندي اثنين Key و اثنين Certificate
واحد عند ال Route والثاني عند ال Pod



كدا ال outside Cluster
وال inside Cluster
كل واحد شغال ب Key مختلفة

OpenShift Route

* openssl req -x509 -newkey rsa:2048 -nodes -keyout myapp-key.pem -out myapp-cert.crt --days 180

- كذا إنشاء Self-signed SSL Cert + Private Key

* oc create deployment myapp --port 8080
--image bitnami/nginx

* oc expose deployment myapp --type NodePort --port 8080

* oc get pods

* oc get service

كذا هنا اننا نلجأ لـ IP + Port وانا عايز اأستخرج URL

* oc expose service myapp

- هيعطيني default URL

* oc get route

- ال URL عبارة عن اسم ال service وال Cluster URL
- فال default Service-NS.Cluster-URL

لما نشتغل بال URL للألف هيعطيني ال HTTP
لو عايز اقلية HTTPS هتجرب آتية القليلة

* oc delete route myapp

* oc get route

* oc create route edge --service myapp --hostname

www.senior.com --key myapp-key.pem --cert myapp-cert-ca

* oc get route

هينظرونا Link هتألف ونضيف بال IP في ملف /etc/hosts